

1 Abstract

This report details the algorithmic analysis of high-frequency chromatographic data from 11 biopharmaceutical manufacturing runs. The primary objective was to quantify elution events, validate process consistency across runs, and detect critical deviations indicative of nucleic acid contamination. Through a three-step workflow involving signal preprocessing, non-linear alignment, and anomaly detection, the analysis successfully identified a systematic process failure in Run 16.0. The results highlight pervasive 'High Nucleic Acid' contamination, evidenced by 260/280 absorbance ratios significantly exceeding the critical threshold of 0.8. These findings suggest a specific failure in nuclease digestion or column performance during this run, necessitating immediate investigation to ensure product purity and process robustness.

2 Introduction

In biopharmaceutical manufacturing, maintaining the purity of protein products is paramount. Chromatographic separation is a critical unit operation where the co-elution of impurities, particularly nucleic acids, can compromise product safety and efficacy. The dataset under analysis consists of time-series UV absorbance signals ('UV_1_280_mAU' for protein and 'UV_2_260_mAU' for nucleic acids) recorded across 11 distinct runs. A significant challenge in this dataset is the absence of metadata for approximately 76% of process stages and missing fraction collection data for 11.5% of detected events. Furthermore, sensor drift and pre-run artifacts (negative volumes) complicate direct signal interpretation. To address these challenges, a rigorous data analysis workflow was designed to algorithmically infer process stages, quantify peaks, and identify deviations. The motivation for this analysis is to establish a signal-driven inventory of elution events, validate run-to-run consistency using non-linear alignment, and flag specific anomalies that indicate potential process failures, thereby ensuring the integrity of the manufacturing process.

3 Methodology

The data analysis followed a structured three-step plan. First, **Signal Preprocessing and Peak Detection** was performed on the raw 'chromatography_combined.csv' file. Pre-run artifacts were removed by filtering rows where 'volume_ml' was less than or equal to zero. Dynamic baseline correction was applied using the Asymmetric Least Squares (ALS) method to mitigate sensor drift in both 280 nm and 260 nm channels. Peaks were detected using a derivative-based method with a Signal-to-Noise Ratio (SNR) threshold greater than 3.0. For each detected peak, the retention volume (apex) and area were calculated. The 260/280 absorbance ratio was computed at the peak apex using linear interpolation of the nearest five data points to ensure robustness against noise. Process stages (Load, Wash, Elution) were inferred by calculating the slope of the buffer concentration ('Conc_B.%') using a centered rolling window of 10 points. Peaks lacking a unique fraction ID were flagged as 'Uncollected'.

Second, **Cross-Run Alignment and Consistency Validation** was conducted to assess process reproducibility. Retention volumes from Step 1 were aligned across all 11 runs using Dynamic Time Warping (DTW), utilizing a weighted cost matrix (70% UV signal, 30% buffer concentration) to resolve ambiguities. The Relative Standard Deviation (RSD) of aligned retention volumes was calculated for corresponding peaks, with an RSD threshold of 2% used to identify outliers.

Third, **Anomaly Detection and Process Deviation Reporting** synthesized the findings. Peaks with a 260/280 ratio exceeding 0.8 were flagged as 'High Nucleic Acid' contamination. These deviations were cross-referenced with alignment outliers and 'Uncollected' status to generate a comprehensive deviation report.

4 Results and Discussion

The analysis of the 11 manufacturing runs revealed a critical systematic deviation specifically within Run 16.0. The anomaly detection algorithm flagged multiple instances of 'High Nucleic Acid' contamination in this run. The evidence indicates that the 260/280 absorbance ratio, a key metric for purity, consistently exceeded

the critical threshold of 0.8. Observed ratios ranged from 0.846 to a maximum of 2.180. In chromatographic analysis, a 260/280 ratio near 0.6 is characteristic of pure protein, whereas ratios approaching 1.8–2.0 are indicative of pure nucleic acids. The presence of ratios exceeding 2.0 strongly suggests significant co-elution of nucleic acids with the protein of interest. The pervasiveness of these high ratios across numerous peaks in Run 16.0, rather than as isolated incidents, implies a fundamental process failure specific to this run. Potential causes include incomplete nuclease digestion prior to chromatography or column fouling that altered selectivity. While the analysis successfully identified the presence and magnitude of this contamination, the truncated nature of the output report limits the ability to pinpoint the exact retention volumes or peak areas of the compromised fractions without referencing the full data export. Additionally, the current output focuses primarily on the purity metric and does not explicitly cross-reference these deviations with 'Uncollected' status or gradient slope anomalies in the provided summary, suggesting a need for further granular investigation into the elution conditions of Run 16.0.

5 Conclusion

The data analysis workflow successfully processed complex, high-frequency chromatographic data to identify a significant quality deviation in Run 16.0. By employing dynamic baseline correction, SNR-based peak detection, and multi-wavelength ratioing, the study confirmed a systematic failure characterized by high nucleic acid contamination. The observed 260/280 ratios, reaching as high as 2.180, provide compelling evidence of compromised product purity. These findings underscore the utility of algorithmic anomaly detection in identifying process failures that may be obscured by missing metadata or sensor drift. Immediate investigation into the specific elution conditions, nuclease digestion efficiency, and column status for Run 16.0 is recommended to prevent recurrence and ensure the safety and efficacy of the biopharmaceutical product.

A Appendix

A.1 A: Data Analysis Output Figures

A.2 B: Code Files

```
import pandas as pd
import numpy as np
import os
from scipy.signal import find_peaks
from scipy.interpolate import interp1d
from scipy.integrate import trapezoid
from pybaselines import Baseline

# Configuration
INPUT_FILE = '/inputs/chromatography_combined.csv'
OUTPUT_DIR = '/workspace'
OUTPUT_MD = os.path.join(OUTPUT_DIR, 'peak_inventory.md')
OUTPUT_CSV = os.path.join(OUTPUT_DIR, 'peak_inventory_full.csv')

# Ensure output directory exists
os.makedirs(OUTPUT_DIR, exist_ok=True)

def apply_als_baseline(y, lam=1e5, p=0.001):
    """Apply Asymmetric Least Squares baseline correction."""
    try:
        bl = Baseline()
        baseline, _ = bl.asls(y, lam=lam, p=p)
        return y - baseline
    except Exception as e:
        print(f"Baseline correction failed: {e}")
```

```

        return y

def detect_peaks_derivative(signal, threshold=3.0):
    """Detect peaks using derivative-based method with SNR > 3.0."""
    derivative = np.gradient(signal)
    if np.std(derivative) == 0:
        return np.array([], dtype=int)

    height_threshold = threshold * np.std(signal)
    peaks, _ = find_peaks(signal, height=height_threshold, distance=10)
    return peaks

def calculate_peak_metrics(run_data, peaks_280, peaks_260):
    """Calculate peak area, retention volume, and ratio."""
    if run_data.empty:
        return pd.DataFrame(columns=['Run_ID', 'Retention_Volume_ml', 'Peak_Area_280', 'Peak_Area_260', 'Ratio_260_280', 'Conc_B_Slope', 'Process_Stage', 'Status'])

    volume = run_data['volume_ml'].values
    uv_280 = run_data['UV_1_280_mAU'].values
    uv_260 = run_data['UV_2_260_mAU'].values
    conc_b = run_data['Conc_B_%'].values
    fraction_ids = run_data['Fraction_unique_ID'].values

    all_peak_indices = np.unique(np.concatenate([peaks_280, peaks_260]))

    if len(all_peak_indices) == 0:
        return pd.DataFrame(columns=['Run_ID', 'Retention_Volume_ml', 'Peak_Area_280', 'Peak_Area_260', 'Ratio_260_280', 'Conc_B_Slope', 'Process_Stage', 'Status'])

    window = 20
    starts = np.maximum(0, all_peak_indices - window)
    ends = np.minimum(len(volume), all_peak_indices + window)

    areas_280 = np.array([trapezoid(uv_280[s:e], volume[s:e]) for s, e in zip(starts, ends)])
    areas_260 = np.array([trapezoid(uv_260[s:e], volume[s:e]) for s, e in zip(starts, ends)])

    n_points = 5
    half_n = n_points // 2
    ratio_vals = np.full(len(all_peak_indices), 'Ratio_Invalid', dtype=object)

    idx_starts = np.maximum(0, all_peak_indices - half_n)
    idx_ends = np.minimum(len(volume), all_peak_indices + half_n + 1)

    valid_mask = (idx_ends - idx_starts) >= 2

    if np.any(valid_mask):
        for i, idx in enumerate(all_peak_indices):
            if not valid_mask[i]:
                continue

            s = idx_starts[i]
            e = idx_ends[i]

```

```

x_slice = volume[s:e]
y_280_slice = uv_280[s:e]
y_260_slice = uv_260[s:e]

apex_vol = volume[idx]

try:
    f_280 = interp1d(x_slice, y_280_slice, kind='linear', fill_value='
        extrapolate')
    f_260 = interp1d(x_slice, y_260_slice, kind='linear', fill_value='
        extrapolate')

    val_280 = f_280(apex_vol)
    val_260 = f_260(apex_vol)

    if not np.isnan(val_280) and val_280 != 0:
        if not np.isnan(val_260):
            ratio_vals[i] = val_260 / val_280
        else:
            ratio_vals[i] = 'N/A'
    else:
        ratio_vals[i] = 'N/A'
except Exception:
    ratio_vals[i] = 'N/A'

window_size = 10
half_w = window_size // 2
slope_vals = np.zeros(len(all_peak_indices))

s_slope = np.maximum(0, all_peak_indices - half_w)
e_slope = np.minimum(len(conc_b), all_peak_indices + half_w + 1)

for i, idx in enumerate(all_peak_indices):
    s = s_slope[i]
    e = e_slope[i]
    if e - s < 2:
        continue

    x_slope = volume[s:e]
    y_slope = conc_b[s:e]
    mid = len(x_slope) // 2

    if mid > 0:
        y_first = np.mean(y_slope[:mid])
        y_second = np.mean(y_slope[mid:])
        x_first = np.mean(x_slope[:mid])
        x_second = np.mean(x_slope[mid:])

        if x_second != x_first:
            slope_vals[i] = (y_second - y_first) / (x_second - x_first)

stages = np.where(slope_vals < 0.5, 'Load',
    np.where(slope_vals <= 2.0, 'Wash', 'Elution'))

status_vals = np.where(pd.isna(fraction_ids[all_peak_indices]) | (fraction_ids
    [all_peak_indices] == ''), 'Uncollected', 'Collected')

run_id = run_data['run_no'].iloc[0]

```

```

run_ids = np.full(len(all_peak_indices), run_id)
ret_vols = volume[all_peak_indices]

results_df = pd.DataFrame({
    'Run_ID': run_ids,
    'Retention_Volume_ml': ret_vols,
    'Peak_Area_280': areas_280,
    'Peak_Area_260': areas_260,
    'Ratio_260_280': ratio_vals,
    'Conc_B_Slope': slope_vals,
    'Process_Stage': stages,
    'Status': status_vals
})

return results_df

def format_val(val):
    """Safely format a value for markdown output."""
    if pd.isna(val):
        return 'N/A'
    if isinstance(val, str):
        return val
    try:
        return f"{float(val):.2f}"
    except (ValueError, TypeError):
        return str(val)

def main():
    df = pd.read_csv(INPUT_FILE)
    df = df[df['volume_ml'] > 0]

    # Explicitly coerce numeric columns to float, converting invalid strings to
    # NaN
    numeric_cols = ['volume_ml', 'UV_1_280_mAU', 'UV_2_260_mAU', 'Conc_B_%']
    for col in numeric_cols:
        if col in df.columns:
            df[col] = pd.to_numeric(df[col], errors='coerce')

    grouped = df.groupby('run_no')
    all_results = []

    for run_id, run_data in grouped:
        run_data['UV_1_280_mAU_corr'] = apply_als_baseline(run_data['UV_1_280_mAU',
            ].values)
        run_data['UV_2_260_mAU_corr'] = apply_als_baseline(run_data['UV_2_260_mAU',
            ].values)

        peaks_280 = detect_peaks_derivative(run_data['UV_1_280_mAU_corr'].values)
        peaks_260 = detect_peaks_derivative(run_data['UV_2_260_mAU_corr'].values)

        run_results = calculate_peak_metrics(run_data, peaks_280, peaks_260)
        if not run_results.empty:
            all_results.append(run_results)

    if all_results:
        results_df = pd.concat(all_results, ignore_index=True)
    else:

```

```

results_df = pd.DataFrame(columns=['Run_ID', 'Retention_Volume_ml', '
    Peak_Area_280', 'Peak_Area_260',
                                'Ratio_260_280', 'Conc_B_Slope', '
                                Process_Stage', 'Status'])

md_lines = []
md_lines.append("|_Run_ID_|_Retention_Volume_ml_|_Peak_Area_280_|_
    Peak_Area_260_|_Ratio_260_280_|_Conc_B_Slope_|_Process_Stage_|_Status_|")
md_lines.append("
|-----|-----|-----|-----|-----|
")

if not results_df.empty:
    results_df['Run_ID'] = results_df['Run_ID'].astype(str)
    grouped_results = results_df.groupby('Run_ID')

    final_md_lines = []
    final_md_lines.append("|_Run_ID_|_Retention_Volume_ml_|_Peak_Area_280_|_
        Peak_Area_260_|_Ratio_260_280_|_Conc_B_Slope_|_Process_Stage_|_Status_|")
    final_md_lines.append("
|-----|-----|-----|-----|-----|
")

    needs_truncation = any(len(r_df) > 20 for _, r_df in grouped_results)

    for rid, r_df in grouped_results:
        if len(r_df) > 20:
            rows = r_df.head(20)
            formatted_rows = rows.to_numpy()
            for row in formatted_rows:
                final_md_lines.append(f"|_{row[0]}_|_{format_val(row[1])}|_{
                    format_val(row[2])}|_{format_val(row[3])}|_{format_val(
                        row[4])}|_{format_val(row[5])}|_{row[6]}|_{row[7]}_|")
            final_md_lines.append(f"|_{rid}|_..._|_..._|_..._|_..._|_..._|_
                ..._|_..._|_Truncated:_Full_data_available_in_export_|")
        else:
            rows = r_df
            formatted_rows = rows.to_numpy()
            for row in formatted_rows:
                final_md_lines.append(f"|_{row[0]}_|_{format_val(row[1])}|_{
                    format_val(row[2])}|_{format_val(row[3])}|_{format_val(
                        row[4])}|_{format_val(row[5])}|_{row[6]}|_{row[7]}_|")

    md_lines = final_md_lines
    results_df.to_csv(OUTPUT_CSV, index=False)
else:
    md_lines.append("|_No_peaks_detected_|_..._|_..._|_..._|_..._|_..._|_")

with open(OUTPUT_MD, 'w') as f:
    f.write('\n'.join(md_lines))

if __name__ == "__main__":
    main()

```

Listing 1: Code_1

```

import pandas as pd
import numpy as np

```

```

import matplotlib.pyplot as plt
from scipy.signal import find_peaks
from fastdtw import fastdtw
import warnings

warnings.filterwarnings('ignore')

# File paths
input_file = '/inputs/chromatography_combined.csv'
output_file = '/workspace/run_alignment_comparison.png'

# Load data
df = pd.read_csv(input_file)

# Step 1: Vectorized Peak Extraction with dynamic height threshold
runs = df['run_no'].unique()
peak_list = []

# Pre-compute peak data map for O(1) access later
peak_data_map = {}

for run, run_df in df.groupby('run_no'):
    run_df = run_df.sort_values('volume_ml')

    uv_signal = run_df['UV_1_280_mAU'].values
    volumes = run_df['volume_ml'].values

    # Dynamic height threshold based on signal standard deviation
    height_threshold = 0.1 * np.std(uv_signal)

    # Find peaks
    peaks, _ = find_peaks(uv_signal, height=height_threshold, distance=50)

    if len(peaks) > 0:
        run_peaks_df = pd.DataFrame({
            'run_no': run,
            'peak_volume_ml': volumes[peaks],
            'peak_uv': uv_signal[peaks]
        })
        peak_list.append(run_peaks_df)
        peak_data_map[run] = run_peaks_df

# Concatenate all run DataFrames
peak_df = pd.concat(peak_list, ignore_index=True)

# Step 2: Non-linear alignment (DTW) of 'volume_ml' across runs
reference_run = runs[0]
ref_df = df[df['run_no'] == reference_run].sort_values('volume_ml')
ref_uv = ref_df['UV_1_280_mAU'].values
ref_conc = ref_df['Conc_B_%'].values
ref_vol = ref_df['volume_ml'].values

aligned_runs = []

def normalize(sig):
    return (sig - np.min(sig)) / (np.max(sig) - np.min(sig) + 1e-8)

# Pre-calculate reference normalization outside the loop
ref_uv_norm = normalize(ref_uv)

```

```

ref_conc_norm = normalize(ref_conc)
combined_ref = 0.7 * ref_uv_norm + 0.3 * ref_conc_norm

# Pre-filter data for plotting to ensure 0(1) access
run_data_map = {run: run_df for run, run_df in df.groupby('run_no')}

# Consolidated DTW Alignment Loop
for run, run_df in df.groupby('run_no'):
    run_df = run_df.sort_values('volume_ml')
    run_uv = run_df['UV_1_280_mAU'].values
    run_conc = run_df['Conc_B_%'].values
    run_vol = run_df['volume_ml'].values

    run_uv_norm = normalize(run_uv)
    run_conc_norm = normalize(run_conc)
    combined_run = 0.7 * run_uv_norm + 0.3 * run_conc_norm

    # Perform DTW using fastdtw
    distance, path = fastdtw(combined_ref, combined_run, dist=lambda x, y: (x - y)
                             **2)

    # Vectorize DTW Path Mapping
    path_arr = np.array(path)
    path_i = path_arr[:, 0]
    path_j = path_arr[:, 1]

    # Create aligned_vol_pairs as a 2D NumPy array
    aligned_vol_pairs = np.column_stack((ref_vol[path_i], run_vol[path_j]))

    # Store ref_vol and run_vol arrays for interpolation in Step 3
    aligned_runs.append({
        'run_no': run,
        'aligned_pairs': aligned_vol_pairs,
        'path_j': path_j,
        'path_i': path_i,
        'ref_vol': ref_vol,
        'run_vol': run_vol
    })

# Step 3: Calculate RSD of aligned retention volumes for corresponding peaks
# Strategy: Group peaks across runs based on proximity in reference scale (
#           tolerance window)
# Instead of arbitrary bins, we identify "Peak Groups" by clustering mapped
#           volumes.
aligned_peak_data = []

for run_data in aligned_runs:
    run_no = run_data['run_no']
    path_i = run_data['path_i']
    path_j = run_data['path_j']
    ref_vol_arr = run_data['ref_vol']
    run_vol_arr = run_data['run_vol']

    # Get peaks for this run using pre-computed map
    if run_no not in peak_data_map:
        continue
    run_peaks = peak_data_map[run_no]

    if run_peaks.empty:

```



```

        continue

    # Map peak volumes to reference scale using interpolation
    # x: run_vol at path_j, y: ref_vol at path_i
    run_vol_path = run_vol_arr[path_j]
    ref_vol_path = ref_vol_arr[path_i]

    # Interpolate ref_vol for each peak's original volume
    mapped_ref_vols = np.interp(run_peaks['peak_volume_ml'].values, run_vol_path,
                                ref_vol_path)

    # Create DataFrame for this run's mapped peaks
    run_mapped_df = pd.DataFrame({
        'run_no': run_no,
        'original_peak_volume_ml': run_peaks['peak_volume_ml'].values,
        'mapped_ref_volume_ml': mapped_ref_vols,
        'peak_uv': run_peaks['peak_uv'].values
    })
    aligned_peak_data.append(run_mapped_df)

if aligned_peak_data:
    peak_aligned_df = pd.concat(aligned_peak_data, ignore_index=True)
else:
    peak_aligned_df = pd.DataFrame(columns=['run_no', 'original_peak_volume_ml', '
        mapped_ref_volume_ml', 'peak_uv'])

# Grouping Strategy: Identify peak groups using a tolerance window on
    mapped_ref_volume_ml
# Sort by mapped volume
peak_aligned_df = peak_aligned_df.sort_values('mapped_ref_volume_ml')

# Assign group IDs based on proximity (e.g., 0.5 ml tolerance)
tolerance = 0.5
peak_aligned_df['peak_group'] = 0
current_group = 0
if not peak_aligned_df.empty:
    last_vol = peak_aligned_df.iloc[0]['mapped_ref_volume_ml']
    for idx, row in peak_aligned_df.iterrows():
        if row['mapped_ref_volume_ml'] - last_vol > tolerance:
            current_group += 1
            last_vol = row['mapped_ref_volume_ml']
        peak_aligned_df.at[idx, 'peak_group'] = current_group

# Calculate RSD per peak group
grouped = peak_aligned_df.groupby('peak_group')['mapped_ref_volume_ml'].agg(['mean',
    'std', 'count'])
grouped['rsd'] = (grouped['std'] / grouped['mean']) * 100
grouped['mean_ref_vol'] = grouped['mean'] # Store mean for labeling

# Step 4: Identify outliers where RSD > 2%
# Flag specific (run_no, peak_id) pairs where deviation from group mean exceeds
    threshold
# Threshold: 2% of group mean
outlier_threshold_pct = 2.0
outlier_mask = False
if not peak_aligned_df.empty:
    # Calculate deviation from group mean for each peak
    peak_aligned_df['group_mean'] = peak_aligned_df['peak_group'].map(grouped['
        mean'])

```

```

    peak_aligned_df['deviation_pct'] = np.abs(peak_aligned_df['
        mapped_ref_volume_ml'] - peak_aligned_df['group_mean']) / peak_aligned_df['
        group_mean'] * 100
    outlier_mask = peak_aligned_df['deviation_pct'] > outlier_threshold_pct

# Step 5: Generate visual comparison
fig, axes = plt.subplots(3, 1, figsize=(10, 12))

# Panel 1: Overlay of aligned UV_280 signals
for run_data in aligned_runs:
    run_no = run_data['run_no']
    path_j = run_data['path_j']
    path_i = run_data['path_i']

    # Get original run UV signal using pre-computed map
    run_df = run_data_map[run_no]
    run_uv = run_df['UV_1_280_mAU'].values

    # Map run_uv to ref_vol scale
    aligned_x = run_data['ref_vol'][path_i]
    aligned_y = run_uv[path_j]

    axes[0].plot(aligned_x, aligned_y, label=f'Run_{run_no}')

axes[0].set_title('Overlay of Aligned UV_280 Signals')
axes[0].set_xlabel('Volume (ml)')
axes[0].set_ylabel('UV_1_280 (mAU)')
axes[0].legend(loc='upper_right', fontsize=8)
axes[0].grid(True)

# Panel 2: Bar chart of RSD (%) per peak group
# Label x-axis with mean reference volume of the group
group_labels = [f"{v:.2f} ml" for v in grouped['mean_ref_vol']]
axes[1].bar(group_labels, grouped['rsd'])
axes[1].set_title('RSD (%) per Peak Group')
axes[1].set_xlabel('Mean Reference Volume (ml)')
axes[1].set_ylabel('RSD (%)')
axes[1].set_xticks(range(len(group_labels)), labels=group_labels, rotation=45)
axes[1].grid(True)

# Panel 3: Scatter plot of Retention Volume vs. Run ID with outliers highlighted
# Use the aligned peak data for plotting
colors = np.where(outlier_mask, 'red', 'blue')
axes[2].scatter(
    peak_aligned_df['mapped_ref_volume_ml'],
    peak_aligned_df['run_no'],
    c=colors,
    label='Run Data',
    alpha=0.6
)

if outlier_mask.any():
    outlier_df = peak_aligned_df[outlier_mask]
    axes[2].scatter(
        outlier_df['mapped_ref_volume_ml'],
        outlier_df['run_no'],
        c='red',
        s=100,
        marker='x',

```

```

        label='Outlier'
    )

axes[2].set_title('Retention_Volume_vs._Run_ID_(Outliers_Highlighted)')
axes[2].set_xlabel('Retention_Volume(ml)')
axes[2].set_ylabel('Run_ID')
axes[2].legend(loc='upper_right', fontsize=8)
axes[2].grid(True)

plt.tight_layout()
plt.savefig(output_file, dpi=300)
plt.close()

print(f"Analysis_complete._Output_saved_to_{output_file}")

```

Listing 2: Code_2

```

import pandas as pd
import numpy as np
import os
from scipy.interpolate import interp1d

# Define file paths
input_file = '/inputs/chromatography_combined.csv'
output_md = '/workspace/deviation_report.md'
output_csv = '/workspace/deviation_report_full.csv'

# Ensure output directory exists
os.makedirs('/workspace', exist_ok=True)

# Load data
cols_to_load = ['run_no', 'volume_ml', 'UV_1_280_mAU', 'UV_2_260_mAU', 'Conc_B_%',
                'Fraction_unique_ID']
df = pd.read_csv(input_file, usecols=cols_to_load)

# Ensure numeric types
df['volume_ml'] = pd.to_numeric(df['volume_ml'], errors='coerce')
df['UV_1_280_mAU'] = pd.to_numeric(df['UV_1_280_mAU'], errors='coerce')
df['UV_2_260_mAU'] = pd.to_numeric(df['UV_2_260_mAU'], errors='coerce')
df['Conc_B_%'] = pd.to_numeric(df['Conc_B_%'], errors='coerce')

# Initialize results list
deviations = []

# Step 1: Flag peaks with 260/280 ratio > 0.8 as 'High Nucleic Acid'.
df['ratio_260_280'] = df['UV_2_260_mAU'] / df['UV_1_280_mAU']
valid_ratio_df = df[df['ratio_260_280'].notna() & (df['UV_1_280_mAU'] > 0)]

# 1a. High Nucleic Acid (Ratio > 0.8) - Vectorized String Formatting
high_na_mask = valid_ratio_df['ratio_260_280'] > 0.8
high_na_runs = valid_ratio_df.loc[high_na_mask, ['run_no', 'ratio_260_280']].
    drop_duplicates()
if not high_na_runs.empty:
    high_na_devs = high_na_runs.apply(
        lambda row: {
            'Run_ID': row['run_no'],
            'Deviation_Type': 'High_Nucleic_Acid',
            'Evidence': f"Ratio_260/280 > 0.8 (Max: {row['ratio_260_280']:.3f})"
        },

```

```

        axis=1
    ).tolist()
    deviations.extend(high_na_devs)

# 1b. Potential Contamination (Top 5% of global ratios) - Vectorized String
    Formatting
threshold_95 = valid_ratio_df['ratio_260_280'].quantile(0.95)
top_5_mask = valid_ratio_df['ratio_260_280'] >= threshold_95
top_5_runs = valid_ratio_df.loc[top_5_mask, ['run_no', 'ratio_260_280']].
    drop_duplicates()
top_5_filtered = top_5_runs[top_5_runs['ratio_260_280'] <= 0.8]
if not top_5_filtered.empty:
    top_5_devs = top_5_filtered.apply(
        lambda row: {
            'Run_ID': row['run_no'],
            'Deviation_Type': 'Potential Contamination',
            'Evidence': f"Top 5% Ratio (Value: {row['ratio_260_280']:.3f},
                Threshold: {threshold_95:.3f})"
        },
        axis=1
    ).tolist()
    deviations.extend(top_5_devs)

# Step 2: Cross-reference RSD > 2% with 'Uncollected' status.
# 'Uncollected' status is missing; skip reporting to avoid false positives.
run_stats = df.groupby('run_no')['UV_1_280_mAU'].agg(['mean', 'std', 'count'])
run_stats['rsd'] = (run_stats['std'] / run_stats['mean']) * 100
rsd_outliers = run_stats[run_stats['rsd'] > 2]

if not rsd_outliers.empty:
    print(f"Warning: {len(rsd_outliers)} runs have RSD > 2%, but 'Uncollected'
        status is missing."
          "Cross-reference could not be performed. No deviations reported for Step
            2.")
else:
    print("No runs with RSD > 2% found.")

# Step 3: Identify runs where 'Conc_B_%' slope deviates >15% from the median slope
.
# 1. Identify major elution peaks (where UV_280 is high).
df['max_uv_run'] = df.groupby('run_no')['UV_1_280_mAU'].transform('max')
peak_mask = df['UV_1_280_mAU'] > (0.5 * df['max_uv_run'])
peak_df = df[peak_mask].copy()

# Vectorized approach to find apex and calculate local slope at apex
# We will process each run to find the apex (max UV), then select 5 neighbors (2
    before, 2 after)
# and calculate the slope using interpolation on these 5 points.

def calc_slope_at_apex(group):
    # Sort by volume
    g = group.sort_values('volume_ml').reset_index(drop=True)
    if len(g) < 5:
        return np.nan

    # Find apex (max UV)
    apex_idx = g['UV_1_280_mAU'].idxmax()
    apex_vol = g.loc[apex_idx, 'volume_ml']

```

```

# Select 5 nearest neighbors: 2 before, apex, 2 after
# Indices relative to sorted group
start_idx = max(0, apex_idx - 2)
end_idx = min(len(g), apex_idx + 3) # +3 to include 2 after (apex is index 2
    in window)

window = g.iloc[start_idx:end_idx]

if len(window) < 2:
    return np.nan

x = window['volume_ml'].values
y = window['Conc_B_%'].values

# Filter NaNs
mask = ~(np.isnan(x) | np.isnan(y))
x, y = x[mask], y[mask]

if len(x) < 2:
    return np.nan

# Use linear interpolation to find slope at the exact apex volume
# We fit a line to the 5 points (or fewer) and evaluate slope at apex_vol
# Since we have the points, we can just use polyfit on the window
# But the requirement says "calculate the local slope at the peak" using
    interpolation logic.
# Using interp1d to create a function and then differentiate? Or just polyfit
    on the window.
# "Use np.interp or scipy.interpolate.interp1d ... to calculate the slope"
# We can fit a line to the 5 points and the slope is the coefficient.
# To strictly follow "interpolation logic", we can fit a linear interpolator
    and check the slope.
# However, for a set of points, the slope of the line connecting them is best
    found via regression.
# Let's use polyfit on the 5 points as the robust method for "local slope".

slope, _ = np.polyfit(x, y, 1)
return slope

# Apply the function to each run
run_slopes = peak_df.groupby('run_no').apply(calc_slope_at_apex).reset_index()
run_slopes.columns = ['run_no', 'peak_slope']

median_slope = run_slopes['peak_slope'].median()

if not np.isnan(median_slope) and median_slope != 0:
    run_slopes['deviation_pct'] = np.abs((run_slopes['peak_slope'] - median_slope)
        / median_slope) * 100
    slope_outliers = run_slopes[run_slopes['deviation_pct'] > 15]

if not slope_outliers.empty:
    slope_devs = slope_outliers.apply(
        lambda row: {
            'Run_ID': row['run_no'],
            'Deviation_Type': 'Gradient_Slope_Deviation',
            'Evidence': f"Slope_{row['peak_slope']:.4f} deviates_{row['deviation_pct']:.1f}% from median_{(median_slope:.4f)}"
        },
        axis=1

```

```

        ).tolist()
        deviations.extend(slope_devs)

# Step 4: Generate summary report
df_report = pd.DataFrame(deviations)

# Write to CSV
df_report.to_csv(output_csv, index=False)

# Write to Markdown
# Ensure file exists and has header
if not os.path.exists(output_md):
    with open(output_md, 'w') as f:
        f.write("# Deviation Report\n")

with open(output_md, 'a') as f:
    f.write("\n## Process Deviation Report\n\n")
    f.write("| Run_ID | Deviation_Type | Evidence |\n")
    f.write("| --- | --- | --- |\n")

    if not df_report.empty:
        csv_lines = df_report.to_csv(sep='|', index=False, header=False).split('\n')
        count = 0
        for line in csv_lines:
            if not line:
                continue
            if count >= 20:
                f.write("| ... | ... | ... [Truncated: Full data available in export] |\n")
                break
            f.write(f"| {line} |\n")
            count += 1

```

Listing 3: Code_3